



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NACC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Applied Machine Learning for Industry Solutions (PECO311C)
**Unit -2 : DATA ANALYSIS AND MACHINE LEARNING TECHNIQUES FOR
INDUSTRIAL APPLICATIONS**

Dr. A. V. Brahmane
Assistant Professor





Contents

- **Exploratory Data Analysis (EDA) Techniques** for summarizing and visualizing datasets to uncover patterns, spot anomalies, and test hypotheses.
- **Feature Engineering and Selection Methods** for creating new features, selecting relevant features, and reducing dimensionality to improve model performance.
- **Supervised Learning Algorithms** Decision Trees, Random Forests, and Gradient Boosting Machines, including their applications in industry.
- **Unsupervised Learning Techniques** K-Means, Hierarchical Clustering, and DBSCAN, and their use cases in industrial scenarios.



Exploratory Data Analysis (EDA) Techniques

Goals of EDA:

Understand data structure and distribution
Identify missing values, outliers, and anomalies
Detect relationships between features
Test basic hypotheses
Choose proper preprocessing steps

EDA is a critical first step in data analysis that involves summarizing the main characteristics of a dataset, often using visual methods.

It helps:

- . Discover patterns and trends.
- . Detect anomalies or outliers.
- . Check assumptions.
- . Formulate hypotheses.

```
import pandas as pd          # For working with data in tables (dataframes)
import numpy as np           # For number operations (like arrays, stats)
import matplotlib.pyplot as plt # For creating basic charts
import seaborn as sns        # For creating beautiful statistical graphs
# For inline plotting in Jupyter
%matplotlib inline

# Set Seaborn style
sns.set(style='whitegrid')
```

Note:The **Titanic dataset** is based on the real-life sinking of the RMS Titanic in 1912. It contains information about passengers on board and whether they survived. It is widely used to teach classification, data cleaning, feature engineering, and model evaluation.

```
# Example: Load the Titanic dataset
df = sns.load_dataset('titanic')

# Preview the data
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

```
df.describe()
```

```
Dataset Shape: (891, 15)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 891 entries, 0 to 890
```

```
Data columns (total 15 columns):
```

#	Column	Non-Null	Count	Dtype
0	survived	891	non-null	int64
1	pclass	891	non-null	int64
2	sex	891	non-null	object
3	age	714	non-null	float64
4	sibsp	891	non-null	int64
5	parch	891	non-null	int64
6	fare	891	non-null	float64
7	embarked	889	non-null	object
8	class	891	non-null	category
9	who	891	non-null	object
10	adult_male	891	non-null	bool
11	deck	203	non-null	category
12	embark_town	889	non-null	object
13	alive	891	non-null	object
14	alone	891	non-null	bool

```
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
```

```
memory usage: 80.7+ KB
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208



Univariate Analysis

Univariate Analysis is the examination of a single variable (column) in a dataset. It helps us understand the distribution, central tendency, and spread of the data.

Goals of Univariate Analysis

Understand the distribution (normal, skewed, etc.) Identify outliers

Measure central tendency (mean, median, mode)

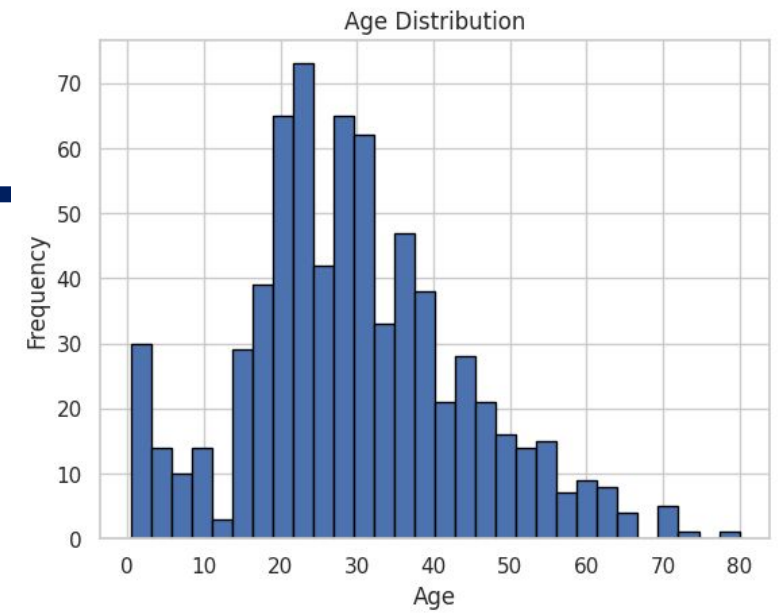
Measure dispersion (variance, standard deviation, range)

Visualize the frequency or density

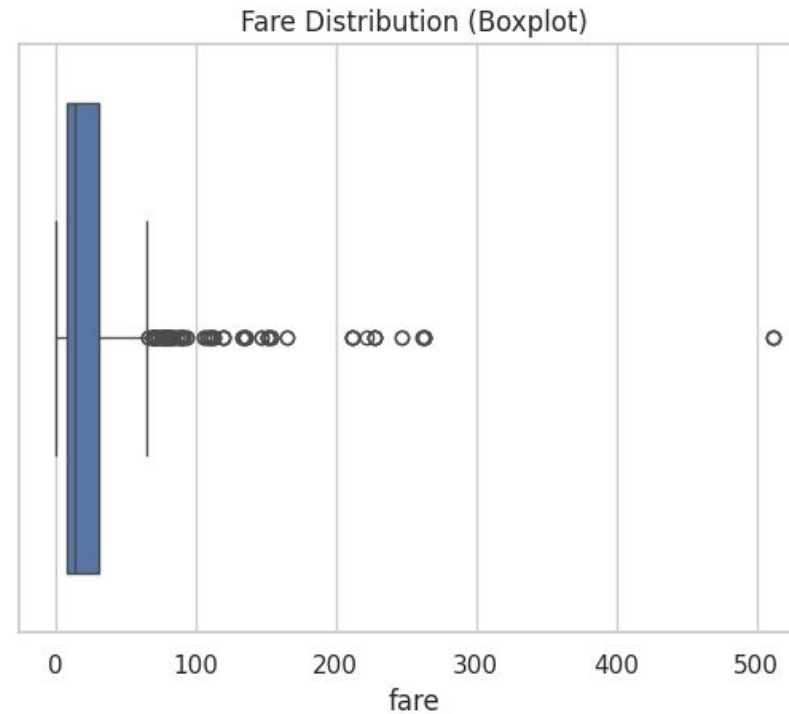


Univariate Analysis

```
#Numerical Features  
# Histogram  
df['age'].hist(bins=30, edgecolor='black')  
plt.title('Age Distribution')  
plt.xlabel('Age')  
plt.ylabel('Frequency')  
plt.show()
```



```
# Boxplot to detect outliers  
sns.boxplot(x=df['fare'])  
plt.title('Fare Distribution (Boxplot)')  
plt.show()
```



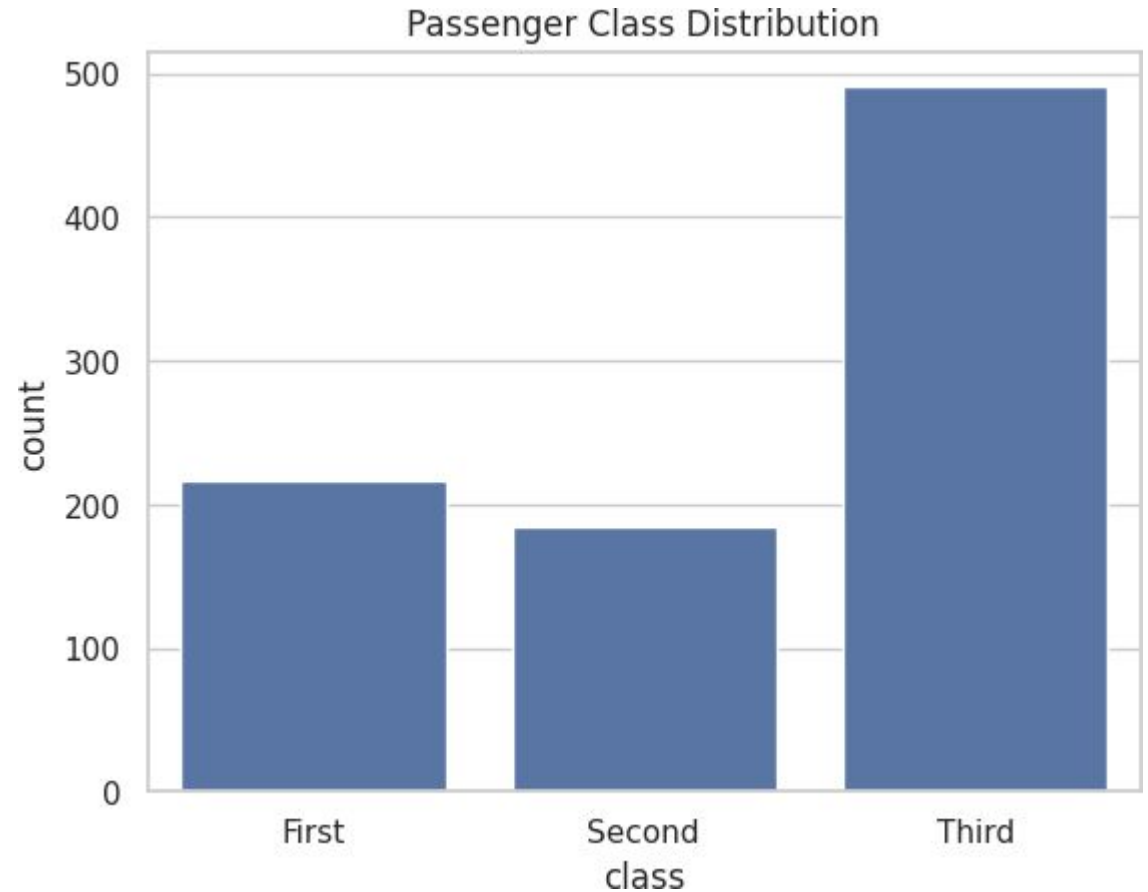


Univariate Analysis

```
#Categorical Features
# Countplot
sns.countplot(x='class', data=df)
plt.title('Passenger Class
Distribution') plt.show()

# Value counts
df['sex'].value_counts(
)
```

	count
Male	577
Female	314





Bivariate Analysis

Bivariate Analysis involves exploring the relationship between **two variables**. It helps identify **correlations, associations, or dependencies**.

Goals of Bivariate Analysis

- Understand how two variables are related

- Detect patterns or trends

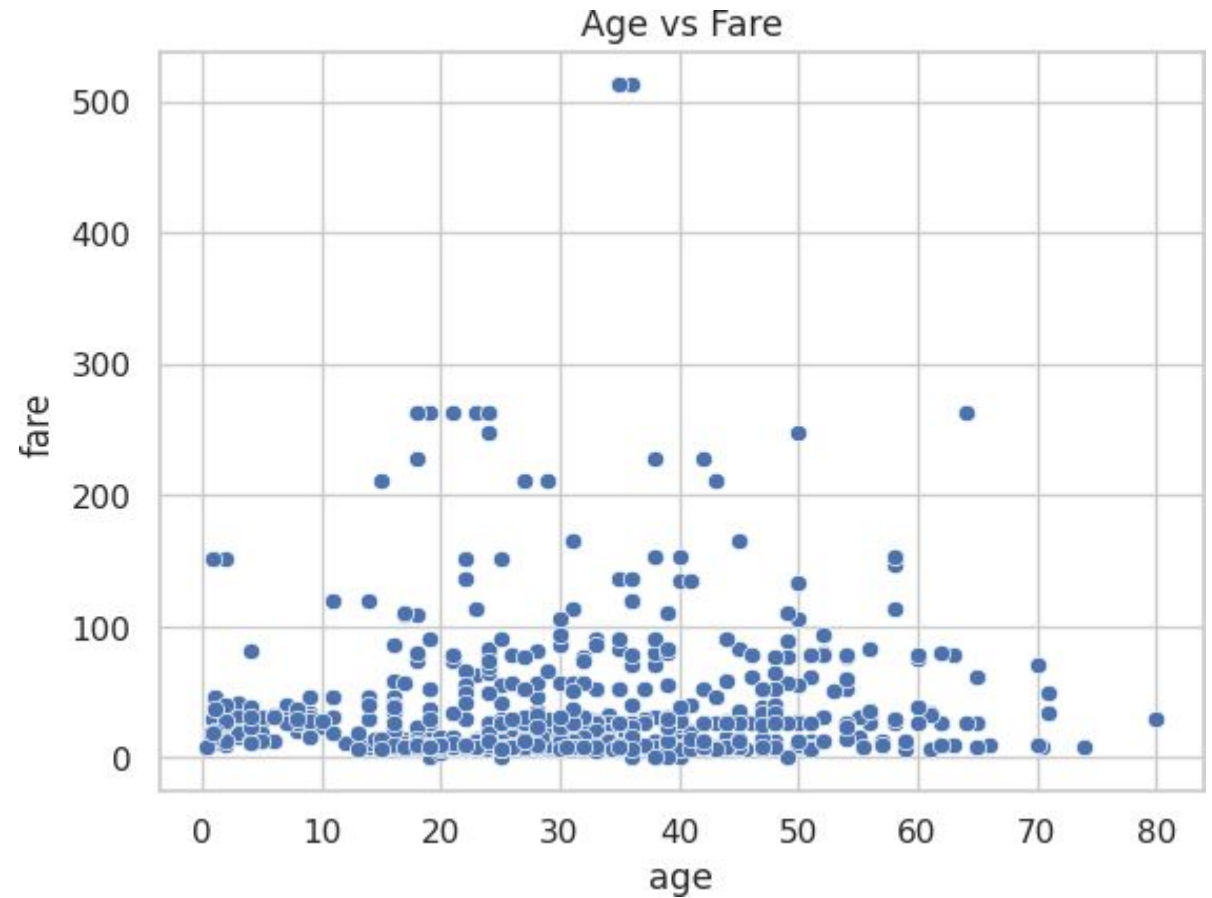
- Support hypothesis generation (e.g., income vs. spending)

- Guide feature selection for modeling



Bivariate Analysis

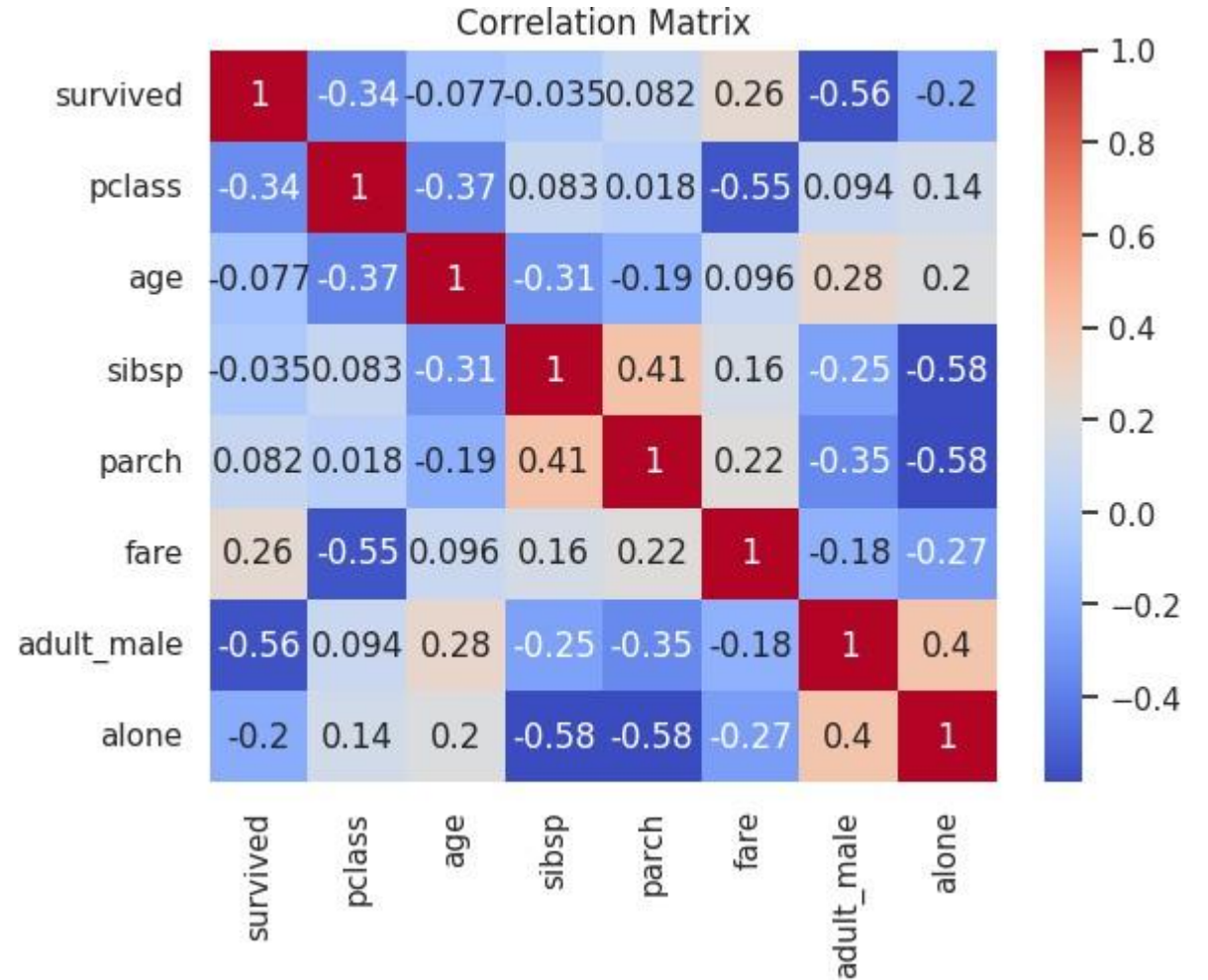
```
#Numerical vs  
Numerical # Scatter  
plot  
sns.scatterplot(x='age',  
y='fare', data=df)  
plt.title('Age vs Fare')  
plt.show()
```





Bivariate Analysis

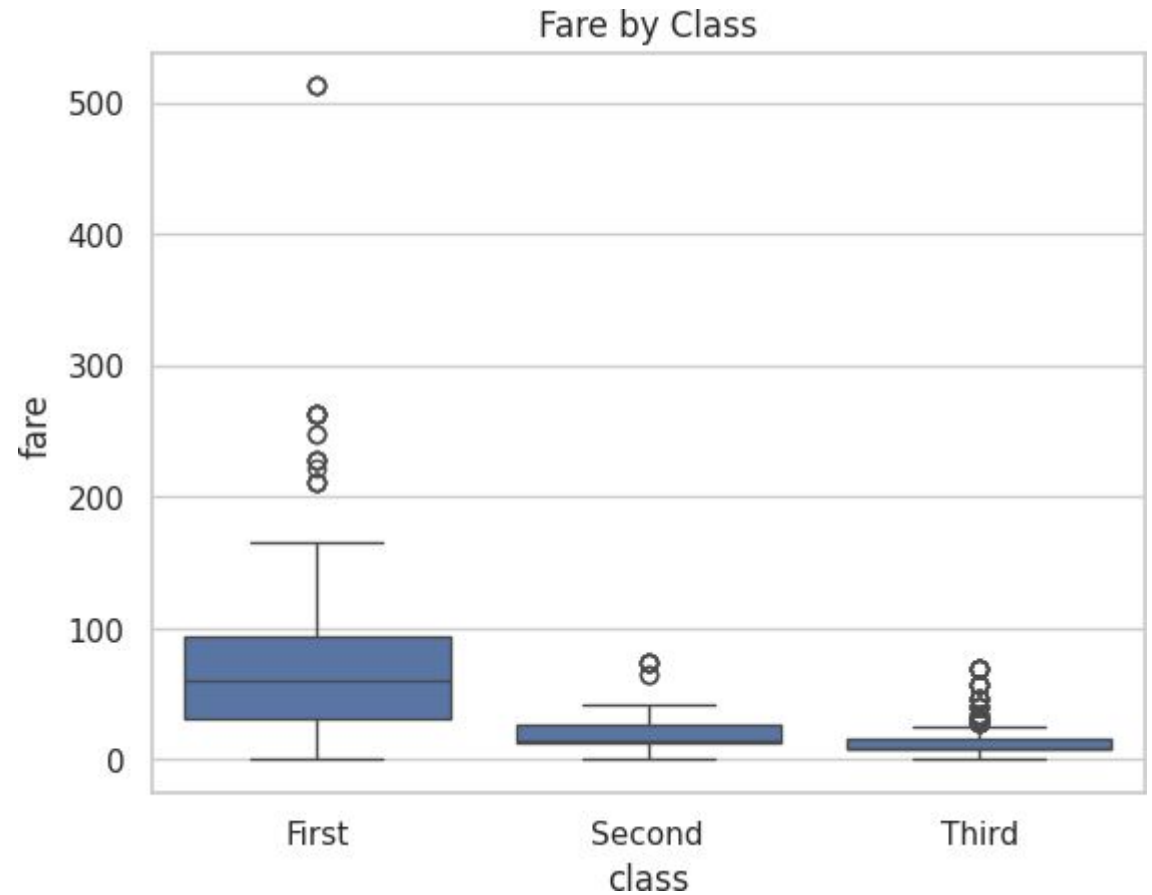
```
# Numerical vs  
Numerical # Correlation  
Matrix  
correlation =  
df.corr(numeric_only=True)  
sns.heatmap(correlation, annot=True,  
cmap='coolwarm')  
plt.title('Correlation  
Matrix') plt.show()  
# -1 means one goes up, the other down  
  
# 0 means no relation
```





Bivariate Analysis

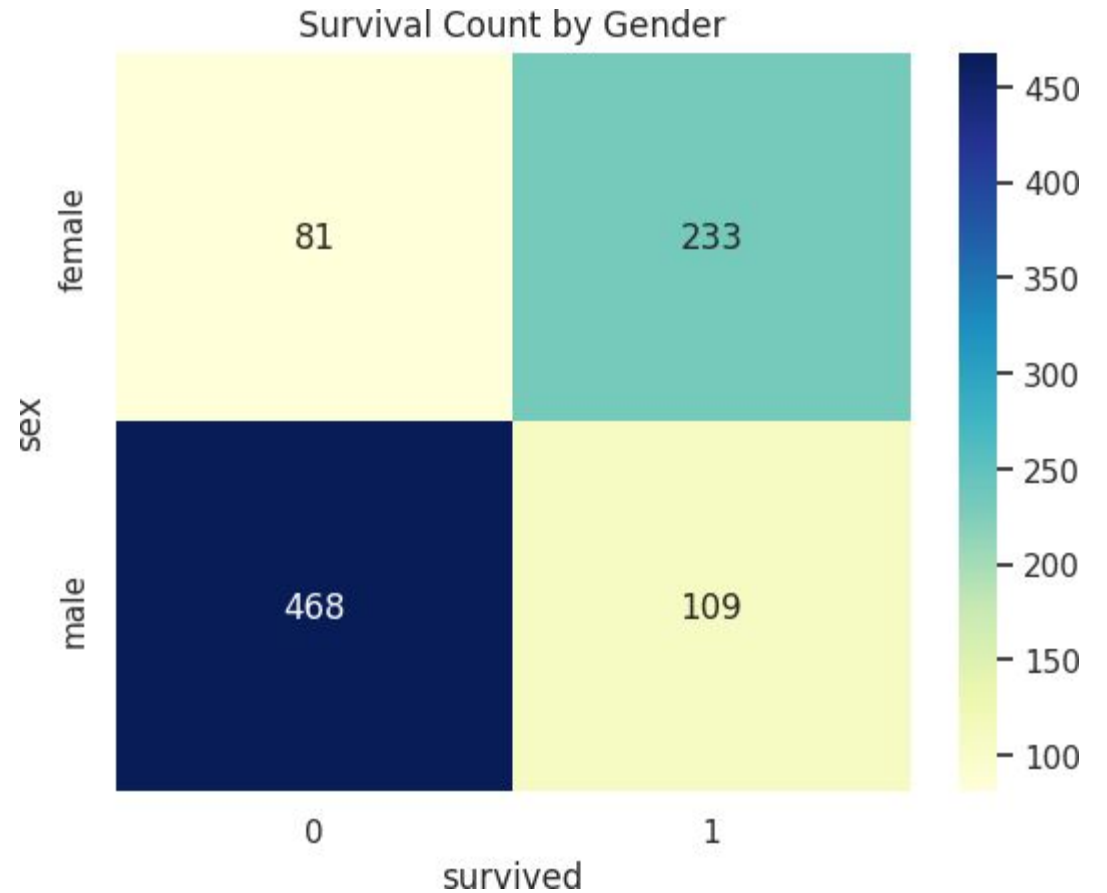
```
#Categorical vs Numerical  
# Boxplot of Fare by  
Class  
sns.boxplot(x='class', y='fare',  
data=df) plt.title('Fare by Class')  
plt.show()  
  
# 1st class passengers pay more than  
3rd class.
```





Bivariate Analysis

```
#Categorical vs  
Categorical # Crosstab and  
heatmap  
ct = pd.crosstab(df['sex'],  
df['survived']) sns.heatmap(ct,  
annot=True, cmap='YlGnBu', fmt='d')  
plt.title('Survival Count by Gender')  
plt.show()
```





Outlier Detection

```
# Using IQR for outlier
detection Q1 =
df['fare'].quantile(0.25) Q3 =
df['fare'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['fare'] < Q1 - 1.5 *
IQR)
| (df['fare'] > Q3 + 1.5 * IQR)]
print(f"Outliers detected:
{outliers.shape[0]}")
```

Outliers detected: 116



Outlier Detection

```
#Outlier Detection
from scipy.stats import zscore
df['z_score'] = zscore(df['fare'])
outliers = df[df['z_score'].abs() > 3]
print(outliers)
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
27	0	1	male	19.0	3	2	263.0000	S	First	
88	1	1	female	23.0	3	2	263.0000	S	First	
118	0	1	male	24.0	0	1	247.5208	C	First	
258	1	1	female	35.0	0	0	512.3292	C	First	
299	1	1	female	50.0	0	1	247.5208	C	First	
311	1	1	female	18.0	2	2	262.3750	C	First	
341	1	1	female	24.0	3	2	263.0000	S	First	
377	0	1	male	27.0	0	2	211.5000	C	First	
380	1	1	female	42.0	0	0	227.5250	C	First	
438	0	1	male	64.0	1	4	263.0000	S	First	
527	0	1	male	NaN	0	0	221.7792	S	First	
557	0	1	male	NaN	0	0	227.5250	C	First	
679	1	1	male	36.0	0	1	512.3292	C	First	
689	1	1	female	15.0	0	1	211.3375	S	First	
700	1	1	female	18.0	1	0	227.5250	C	First	
716	1	1	female	38.0	0	0	227.5250	C	First	
730	1	1	female	29.0	0	0	211.3375	S	First	
737	1	1	male	35.0	0	0	512.3292	C	First	
742	1	1	female	21.0	2	2	262.3750	C	First	
779	1	1	female	43.0	0	1	211.3375	S	First	



Hypothesis Testing

Hypothesis Testing Hypothesis testing is a statistical method to decide if there is enough evidence to infer that a certain condition is true for a population.

In this context: **Null hypothesis** (H_0): The mean fare paid by males and females is the same. **Alternative**

hypothesis (H_1): The mean fare paid by males and females is different.

t-statistic: Quantifies the difference between the sample means. **p-value**: Probability of observing such a difference (or more extreme) if the null hypothesis is true.

Interpreting Output: If $p\text{-value} < 0.05$ (common threshold), reject the null hypothesis: there is a statistically significant difference in mean fare between males and females.

If $p\text{-value} \geq 0.05$, fail to reject the null: no strong evidence of a difference



Hypothesis Testing

```
from scipy.stats import ttest_ind
# Example: Test if mean fare is significantly different between male and female

male_fare = df[df['sex'] == 'male']['fare'].dropna()

female_fare = df[df['sex'] == 'female']['fare'].dropna()

t_stat, p_val = ttest_ind(male_fare, female_fare)

print(f"T-statistic: {t_stat}, P-value: {p_val}")
```

T-statistic: -5.529140269385719, P-value: 4.2308678700429995e-08



Feature Engineering

Feature Engineering and Selection Methods for creating new features, selecting relevant features, and reducing dimensionality to improve model performance.

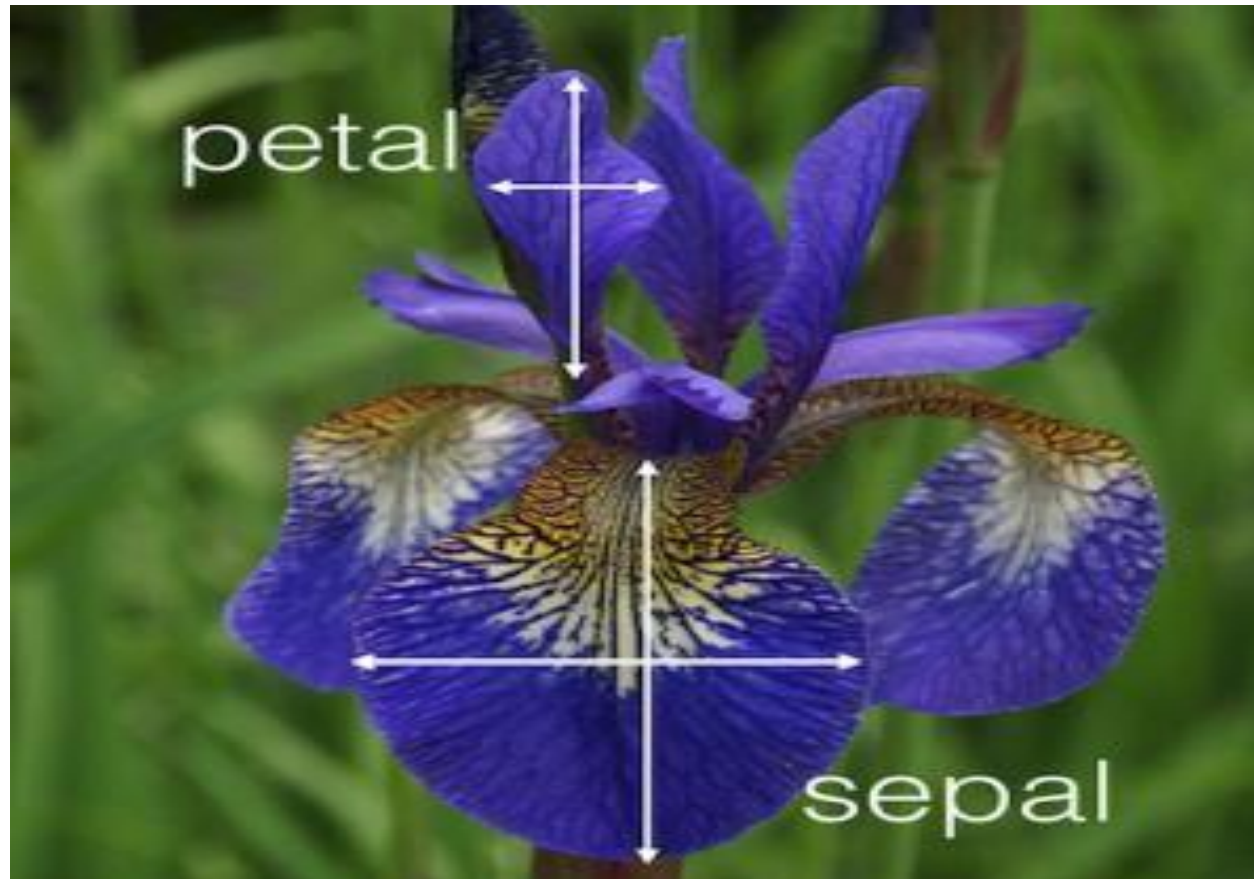
Machine learning models are only as good as the data they're fed. Better features often outperform more complex models.

Feature selection identifies the most relevant features and removes redundant or irrelevant ones.

iris flower Dataset

50 samples of 3 different species of iris (150 samples total)

Measurements: sepal length, sepal width, petal length, petal width





Step by Step PCA

- To understand the value of using PCA for data visualization, the **first part basic visualization of the IRIS dataset after applying PCA.**
- The **second part uses PCA to speed up a machine learning algorithm (logistic regression) on the MNIST dataset.**
- matplotlib is a python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. matplotlib can be used in Python scripts,



Step by Step PCA

- To understand the value of using PCA for data visualization, the **first part basic visualization of the IRIS dataset after applying PCA.**
- The **second part uses PCA to speed up a machine learning algorithm (logistic regression) on the MNIST dataset.**
- matplotlib is a python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. matplotlib can be used in Python scripts,



Step by Step PCA

```
import pandas as pd
import numpy as np
import seaborn as
sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler,
LabelEncoder from sklearn.feature_selection import
SelectKBest, f_classif from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier

# Load the Iris dataset
iris = load_iris()
# Convert to DataFrame
df = pd.DataFrame(data=iris.data,
columns=iris.feature_names) df['species'] = iris.target
# Check the data
df.head()
```



Step by Step PCA

SelectKBest is a feature selection method in scikit-learn that selects the top k features with the highest scores as measured by a statistical test. When used with the scoring function `f_classif`, SelectKBest employs the ANOVA F-value between each feature and the target variable to rank features. The formula for the F-statistic is:

F = MSB – Mean Square Between groups / MSW – Mean Square Within groups

$$MSB = \frac{\sum n_i (\bar{x}_i - \bar{x})^2}{k - 1}$$

Where:

- n_i : number of samples in class i
- $\bar{x}_i$: mean value of feature in class i
- $\bar{x}$: overall mean of that feature
- k : number of classes (here, 3 flower species)

$$MSW = \frac{\sum \sum (x_{ij} - \bar{x}_i)^2}{n - k}$$

Where:

- x_{ij} : value of sample j in class i
- $\bar{x}_i$: mean of class i
- n : total number of samples
- k : number of classes



Step by Step PCA

```
# SelectKBest (Statistical Test)
X = df_scaled.drop('species',
axis=1) y = df_scaled['species']
```

```
selector = SelectKBest(score_func=f_classif,
k='all') fit = selector.fit(X, y)
```

```
# Show feature scores
```

```
scores = pd.DataFrame({'Feature': X.columns, 'Score':
fit.scores_}) scores.sort_values(by='Score', ascending=False)
```

	Feature	Score
2	petal_length	1180.161182
3	petal_width	960.007147
0	sepal_length	119.264502
1	sepal_width	49.160040



Create New Features

```
# Create New Features
# Example: Petal Area and Sepal Area
df['petal_area'] = df['petal_length']
* df['petal_width']
df['sepal_area'] = df['sepal_length']
* df['sepal_width']
df.head(2)
#These new features might help because
area could be more informative than length
or width alone.
```

	sepal_length	sepal_width	petal_length	petal_width	species	petal_area	sepal_area
0	5.1	3.5	1.4	0.2	0	0.28	17.85
1	4.9	3.0	1.4	0.2	0	0.28	14.70
2	4.7	3.2	1.3	0.2	0	0.26	15.04
3	4.6	3.1	1.5	0.2	0	0.30	14.26
4	5.0	3.6	1.4	0.2	0	0.28	18.00



```
# Dimensionality Reduction with PCA & PCA Projection to
```

```
2D pca = PCA(n_components=2)
```

```
principalComponents = pca.fit_transform(X)
```

```
principalDf = pd.DataFrame(data =
```

```
principalComponents
```

```
    , columns = ['principal component 1', 'principal component  
2']) print(principal
```

	principal component 1	principal component 2
0	-2.264703	0.480027
1	-2.080961	-0.674134
2	-2.364229	-0.341908
3	-2.299384	-0.597395
4	-2.389842	0.646835
..	...	...
145	1.870503	0.386966
146	1.564580	-0.896687
147	1.521170	0.269069
148	1.372788	1.011254
149	0.960656	-0.024332

```
[150 rows x 2 columns]
```

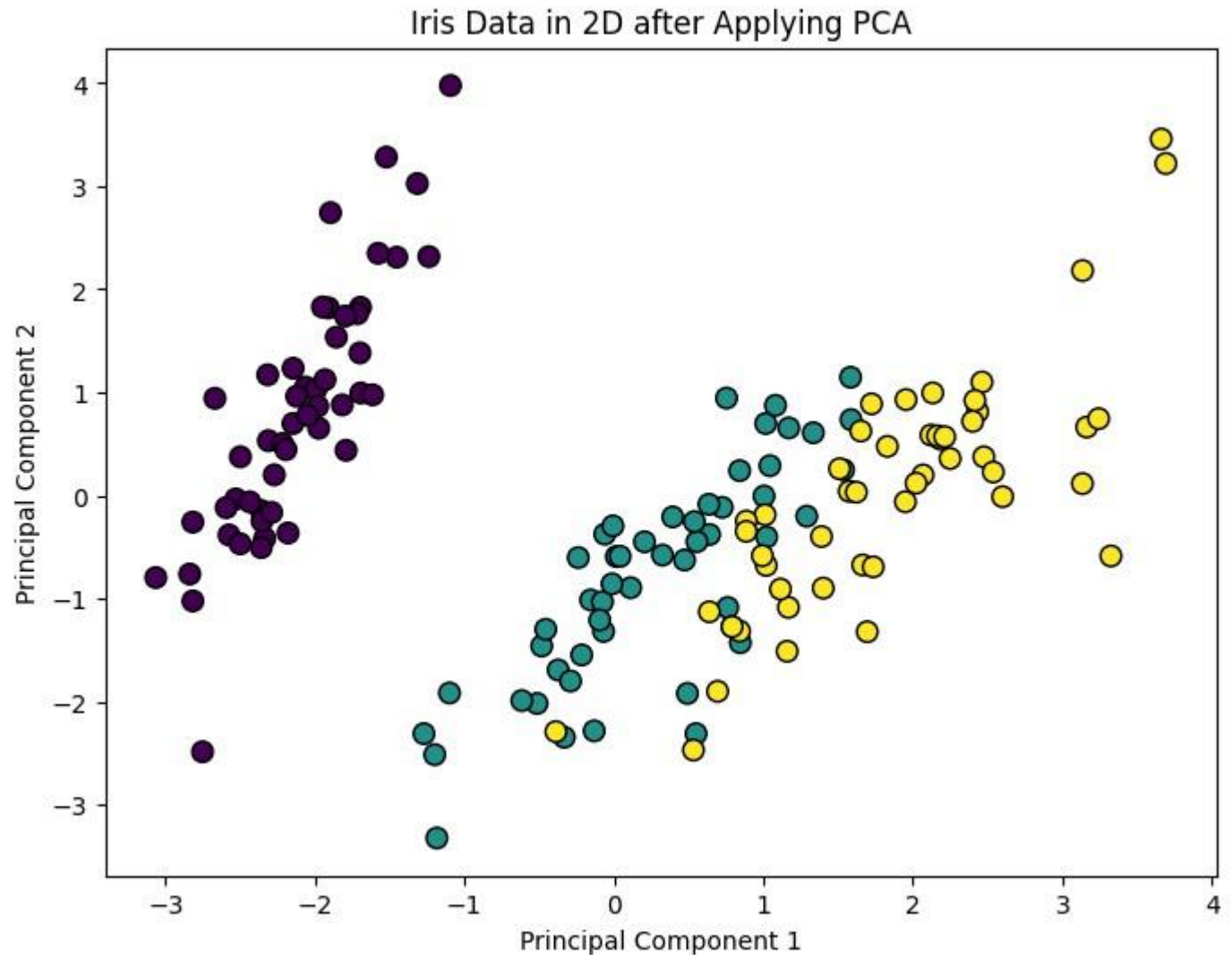


```
# Plot in 2D
```

```
plt.figure(figsize=(8,6))
plt.scatter(principalDf['principal component 1'],
principalDf['principal component 2'], c=y,
cmap='viridis', edgecolor='k', s=70)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Iris Data in 2D after Applying PCA")
plt.show()
pca.explained_variance_ratio_
```

Explained Variance

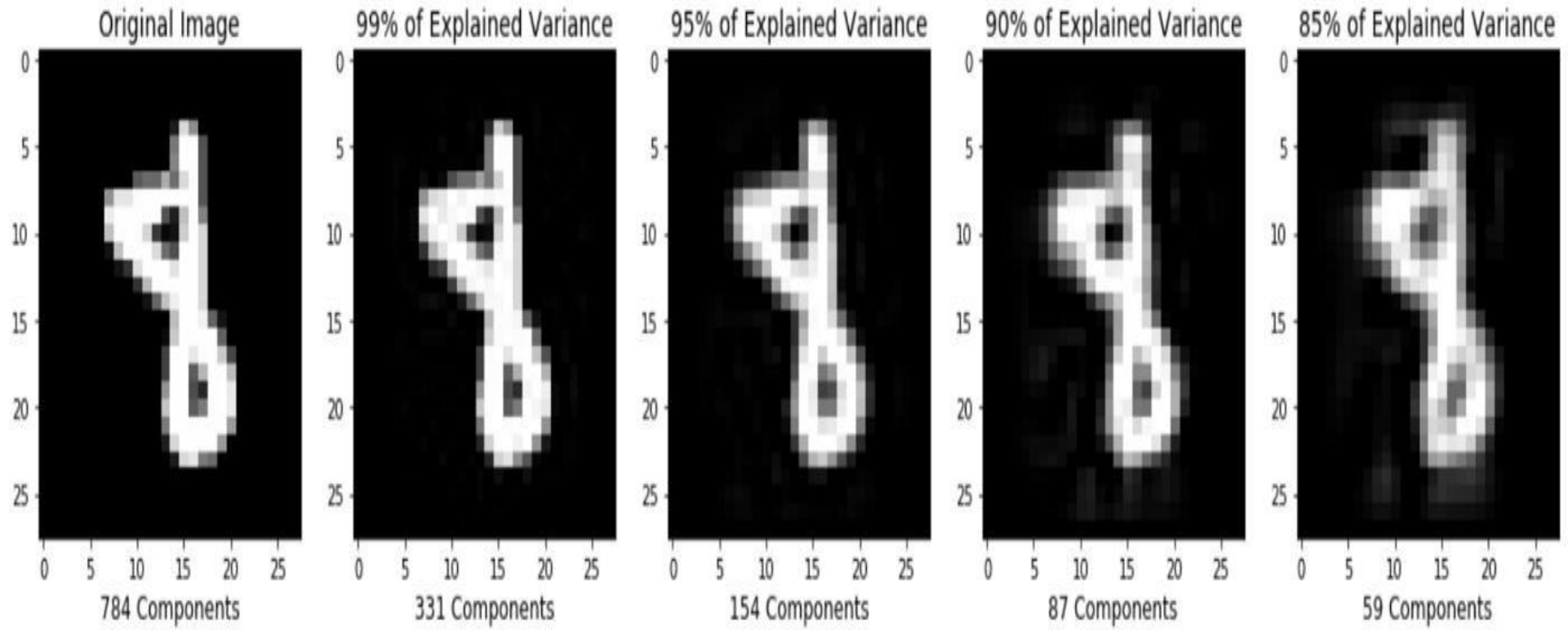
The explained variance tells you how much information (variance) can be attributed to each of the principal components. This is important as while you can convert 4 dimensional space to 2 dimensional space, you lose some of the variance (information) when you do this. By using the attribute `explained_variance_ratio_`, you can see that the **first principal component contains 62.4.%** of the variance and the **second principal component contains 34.0%** of the variance. **Together, the two components contain 96.40% of the information.**





PCA to Speed-up Machine Learning Algorithms

- One of the most important applications of PCA is for speeding up machine learning algorithms. Using the IRIS dataset would be impractical here as the dataset only has 150 rows and only 4 feature columns. The MNIST database of handwritten digits is more suitable as it has 784 feature columns (784 dimensions), a training set of 60,000 examples, and a test set of 10,000 examples.





Supervised Learning Algorithms Decision Trees, Random Forests, and Gradient Boosting Machines, including their applications in industry.

Decision Tree is a decision-making tool that uses a flowchart-like tree structure or is a model of decisions and all of their possible results, including outcomes, input costs and utility.

Decision-tree algorithm falls under the category of supervised learning algorithms. It works for both continuous as well as categorical output variables.

The branches/edges represent the result of the node and the nodes have either:

1. Conditions [Decision Nodes]
2. Result [End Nodes]

A decision tree can be visualized. A decision tree is one of the many Machine Learning algorithms.



Aim: Implement Decision Tree on given Problem

A dataset collected in a cosmetics shop showing details of customers and whether or not they responded to a special offer to buy a new lip-stick is shown in table below. Use this dataset to **build a decision tree**, with Buys as the target variable, to help in buying lip-sticks in the future. **Find the root node of decision tree**. According to the decision tree you have made from previous training data set, what is the decision for the test data: **[Age < 21, Income = Low, Gender = Female, Marital Status = Married]**?

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No



Step1:Compute the Entropy for the data set

Out of 14 instances we have 9 YES and 5 NO

So we have the formula,

$$E(S) = -P(\text{Yes}) \log_2 P(\text{Yes}) - P(\text{No}) \log_2 P(\text{No})$$

$$E(S) = - (9/14)^* \log_2 9/14 - (5/14)^* \log_2 5/14$$

$$E(S) = 0.41 + 0.53 = 0.94$$

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No

Which node to select as root node?

Age

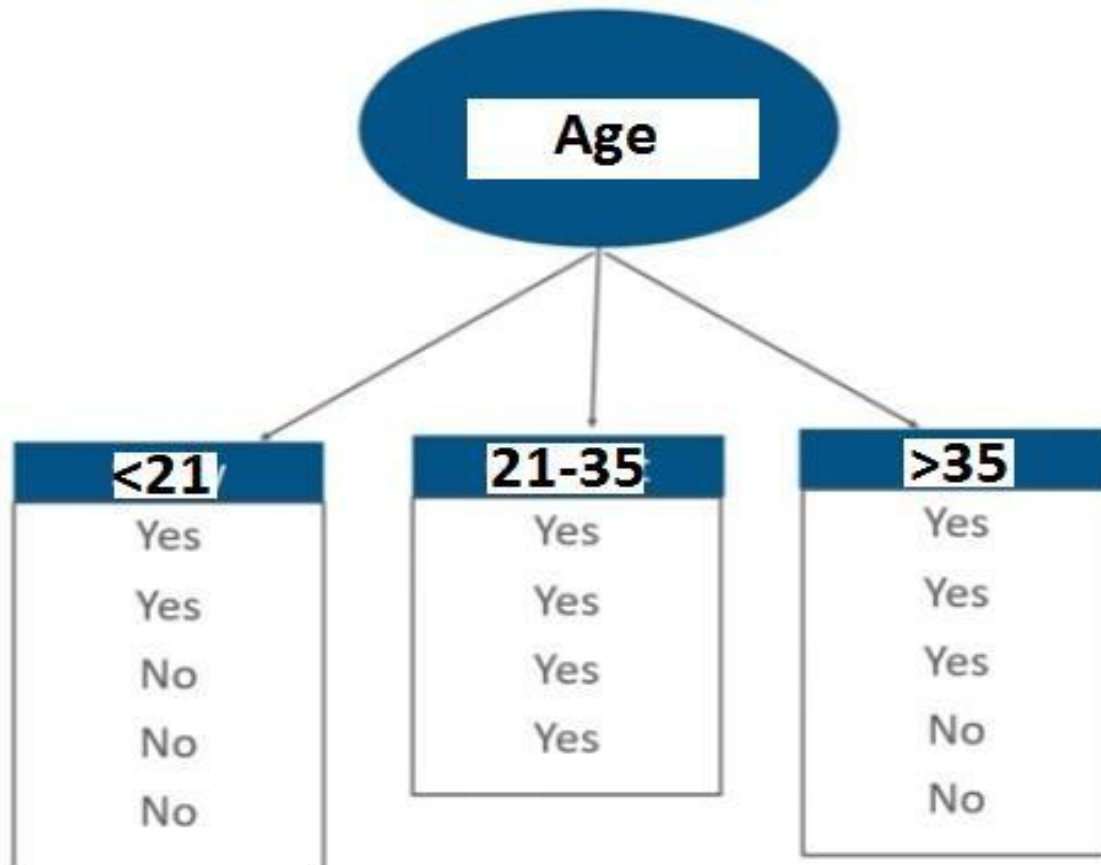
Income

gender

Marital
Status

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No

Which node to select as root node?



ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No



Which node to select as root node?

$$E(\text{age} = <21) = -2/5 \log_2 2/5 - 3/5 \log_2 3/5 = 0.971$$

$$E(\text{age} = 21-35) = -1 \log_2 1 - 0 \log_2 0 = 0$$

$$E(\text{age} = >35) = -3/5 \log_2 3/5 - 2/5 \log_2 2/5 = 0.971$$

Information from outlook,

$$I(\text{age}) = 5/14 \times 0.971 + 4/14 \times 0 + 5/14 \times 0.971 = 0.693$$

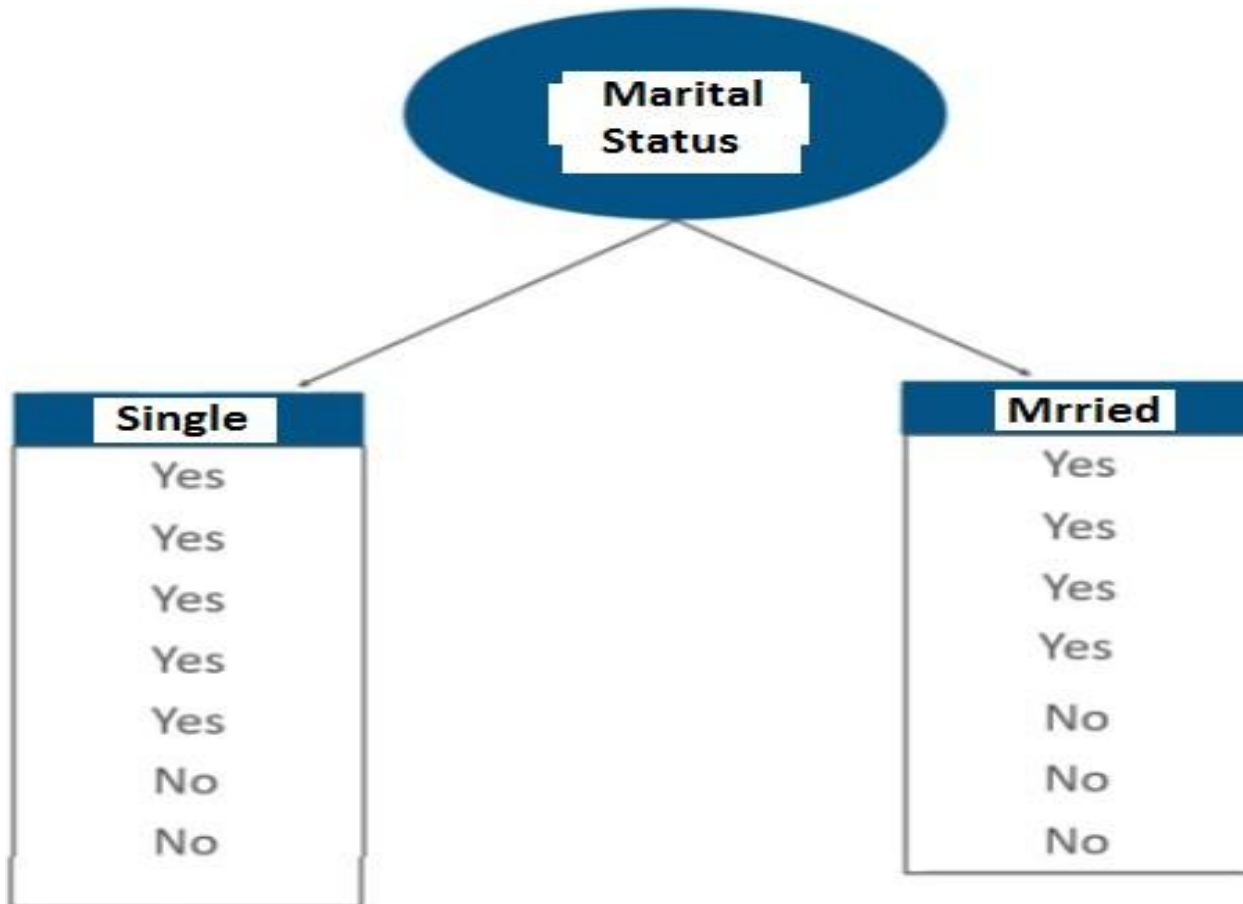
Information gained from age

$$\text{Gain}(\text{age}) = E(S) - I(\text{age})$$

$$0.94 - 0.693 = 0.247$$

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No

Which node to select as root node?



ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No



Which node to select as root node?

$$E(\text{Marital} = \text{True}) = 1$$

status

$$E(\text{Marital} = \text{False}) = 0.811$$

status

Information from windy,

$$I(\text{Marital Status}) = 8/14 \times 0.811 + 6/14 \times 1 = 0.892$$

Information gained from outlook,

$$\text{Gain}(\text{Marital Status}) = E(S) - I(\text{Marital Status})$$

$$0.94 - 0.892 = 0.048$$

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No

With similar Calculations we get

Gain (Age) = 0.247 (root) Gain(Income)= 0.029 Gain(Gender)= 0.024 Gain(Marital Status)=0.048



Explore practical Implementation

Source Code :

Import packages

import numpy as np

import matplotlib.pyplot as

plt import pandas as pd

#reading Dataset

dataset=pd.read_csv("data.csv

") X=dataset.iloc[:, :-1]

y=dataset.iloc[:,5].values

#Perform Label encoding

from sklearn.preprocessing import

LabelEncoder labelencoder_X = LabelEncoder()

X = X.apply(LabelEncoder().fit_transform)

print (X)

from sklearn.tree import

DecisionTreeClassifier

regressor=DecisionTreeClassifier()

regressor.fit(X.iloc[:,1:5],y)

#Predict value for the given expression

X_in=np.array([0,0,0,1])

y_pred=regressor.predict([X_in

]) print ("Prediction:", y_pred)

from sklearn.externals.six import StringIO

from IPython.display import Image

from sklearn.tree import export_graphviz

import pydotplus

Create DOT data

dot_data = StringIO()

export_graphviz(regressor, out_file=dot_data, filled=True,
rounded=True, special_characters=True)

Draw graph

graph = pydotplus.graph_from_dot_data(dot_data.getvalue())

graph.write_png('Decision_Tree.png')

Show graph

Image(graph.create_png())



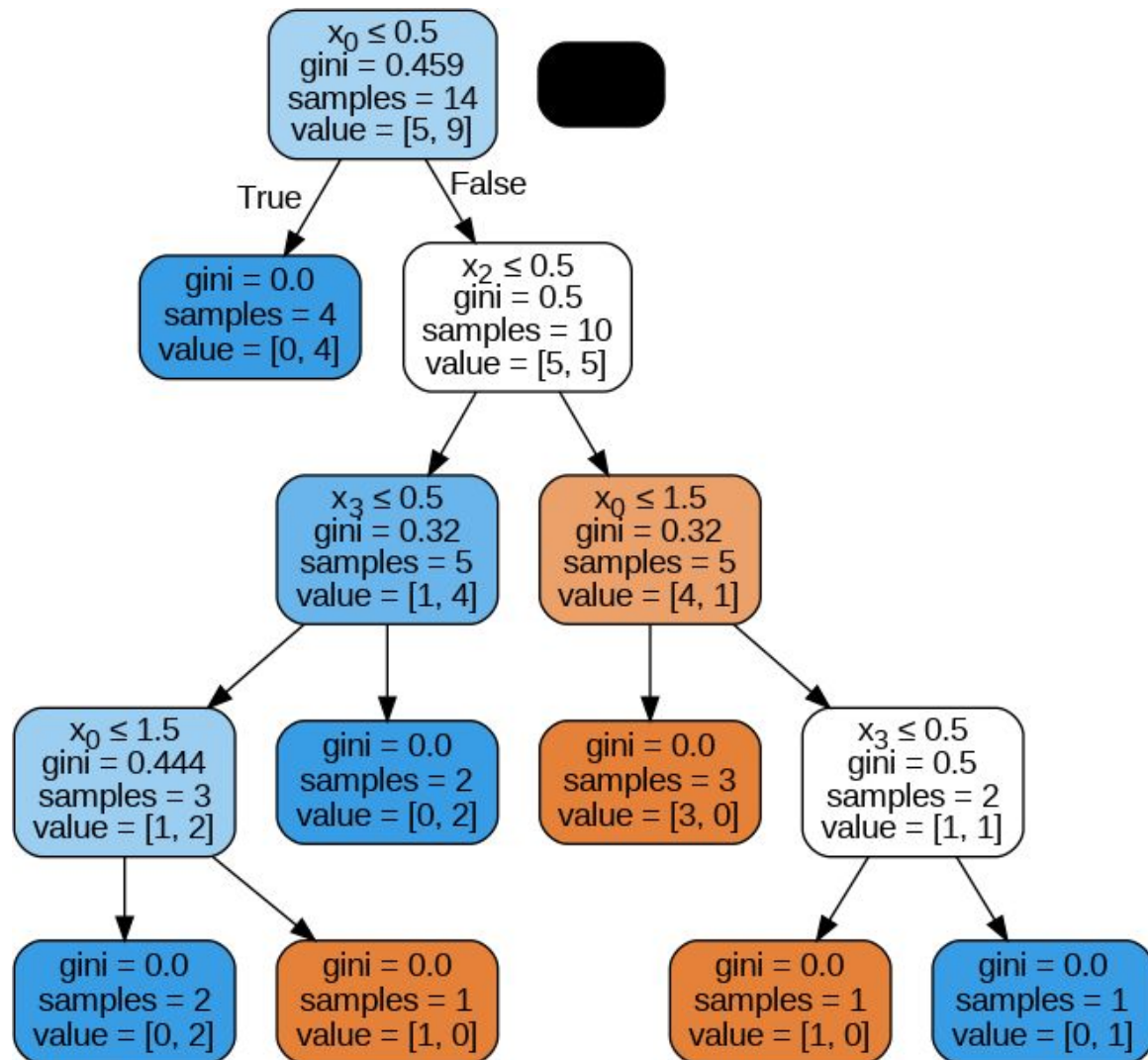
Expected Output

	id	age	income	gender	marital_status
0	0	1	0	1	1
1	1	1	0	1	0
2	2	0	0	1	1
3	3	2	2	1	1
4	4	2	1	0	1
5	5	2	1	0	0
6	6	0	1	0	0
7	7	1	2	1	1
8	8	1	1	0	0
9	9	2	2	0	1
10	10	1	2	0	0
11	11	0	2	1	0
12	12	0	0	0	1
13	13	2	2	1	0

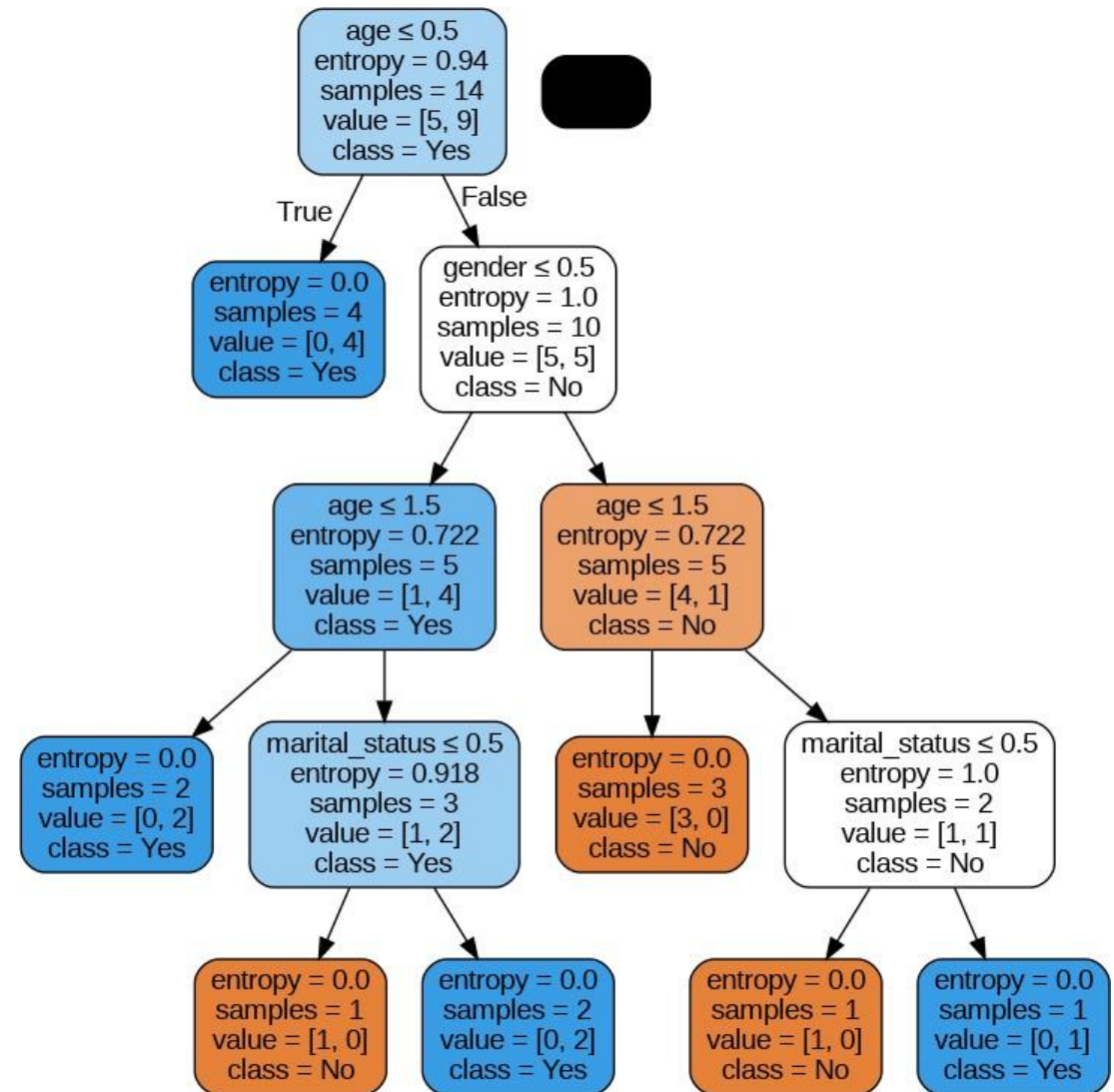
Prediction: ['Yes']



Decision_Tree_GiniIndex



Decision_Tree_InfoGain





Decision Trees

Advantages

- Decision trees are powerful and popular tools for classification and prediction.
- Simpler and ease of use.
- They are able to handle both numerical and categorical attributes.
- Easy to understand.
- State is recorded in memory.
- Provide a clear indication of which fields are most important for prediction or classification



Decision Industrial Applications:

Healthcare: Predicting patient readmission, disease diagnosis, and identifying genetic markers.

Finance: Credit scoring, option pricing, risk assessment, and fraud detection.

Marketing: Customer segmentation, churn prediction, and targeted marketing campaigns.

Business Strategy: Visualization of decision processes and cost-effectiveness analyses



Random Forests

Random forests combine multiple decision trees (They are an ensemble method) to improve predictive accuracy and control overfitting. They are widely used for classification and regression tasks

Key Concepts of Random Forests:

Decision Trees:

Each model in the forest is a decision tree.

A decision tree splits data based on feature values to make predictions.

Ensemble of Trees:

A random forest builds **many decision trees** and combines their predictions.

For classification: it uses **majority voting** (most common class).

For regression: it averages the predictions.

Randomness:

Bootstrap Aggregating (Bagging): Each tree is trained on a random sample (with replacement) of the training data.

Random Feature Selection: When splitting nodes, only a random subset of features is considered.



Random Forests

- **Advantages:**

- High accuracy.
- Resistant to overfitting compared to individual decision trees.
- Works well with both categorical and numerical data.
- Handles missing values and maintains accuracy with unbalanced data.

- **Disadvantages:**

- Slower to train and predict than single decision trees.
- Less interpretable than a single decision tree.
- Large models can consume a lot of memory.



Random Forests

```
Import packages
import numpy as np
import pandas as
pd
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import
RandomForestClassifier from IPython.display import
Image
from sklearn.tree import
export_graphviz import pydotplus
from six import StringIO

# Load dataset
url =
'https://drive.google.com/uc?export=download&id=1jq12mwV15BCFeX52G1\_PGSCr8Y4jLdn8f'
df = pd.read_csv(url)
```



Random Forests

```
# Features and target
X = df.iloc[:, :-1]
y = df.iloc[:, -1].values # Target column
# Encode categorical columns
X = X.apply(LabelEncoder().fit_transform)
```

Create Random Forest Classifier

```
rf_model = RandomForestClassifier(n_estimators=10, criterion='entropy', random_state=0)
rf_model.fit(X.iloc[:, 1:5], y)
```

Predict a value using Random Forest

```
X_in = np.array([0, 1, 0, 1])
y_pred = rf_model.predict([X_in])
print("Prediction:", y_pred)
```

Feature Importance (optional & useful)

```
importances = rf_model.feature_importances_
feature_names = X.columns[1:5]
print("\n Feature Importances:")
for name, importance in zip(feature_names, importances):
    print(f"{name}: {importance:.4f}")
```

```
O/p: Feature
age: 0.4413:
income: 0.1263
gender: 0.2530
marital_status:
0.1795
```



Random Forests Industrial Applications:

Insurance: Risk assessment of customers and claims.

Banking/Finance: Credit scoring, fraud detection, loan default prediction, stock forecasting.

Healthcare: Disease prediction using genetic and imaging data.

Marketing: Customer segmentation, predictive marketing, recommendation systems.

Text/Image Classification: Spam detection, document or message categorization, image recognition



Gradient Boosting Machines:

Gradient Boosting Machines (GBMs) are another powerful ensemble learning technique used for **classification, regression**, and even ranking tasks. They're part of the **boosting** family—unlike Random Forests (which use bagging), GBMs build trees **sequentially**, each one improving on the errors of the previous

When to Use Gradient Boosting? When accuracy is more important than training speed. For tabular datasets where trees work well.

Gradient Boosting Machines create an ensemble of weak prediction models (typically decision trees) using a boosting technique that focuses on improving residual errors from prior models. This delivers high predictive accuracy and robust performance.



Gradient Boosting Machines:

How Gradient Boosting Works:

Start with a weak model, typically a shallow decision tree.

Compute residuals (errors between the true and predicted values).

Train a new model on those residuals to correct the errors.

Add the new model to the ensemble with a weight (learning rate).

Repeat the process for a set number of iterations or until convergence.



Gradient Boosting Machines:

Pros:

- Often **more accurate** than Random Forests for structured/tabular data.
- **Flexible:** Works with various loss functions.
- Can handle **mixed data types**, missing values, and categorical variables (depending on implementation).

Cons:

- **Slower to train** than Random Forests.
- **Sensitive to hyperparameters** (learning rate, tree depth, etc.).
- **Easier to overfit** if not tuned carefully.
- **Sequential** training makes parallelization harder (though libraries like XGBoost, LightGBM, CatBoost optimize this).



Gradient Boosting Machines:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import
GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Load dataset
url =
"https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Cus
[7043 rows x 21
df = pd.read_csv(url) columns]
```

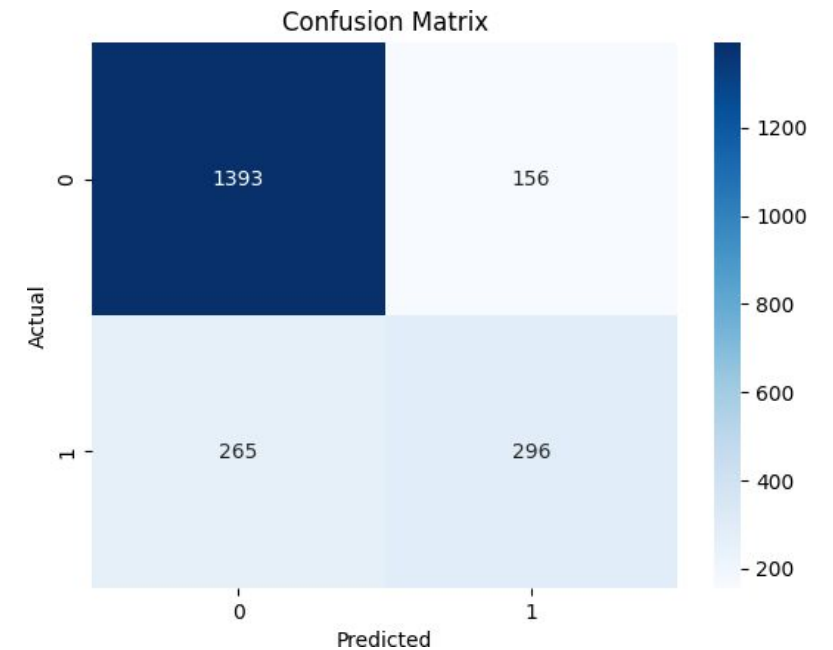


Gradient Boosting Machines:

```
X_train, X_test, y_train, y_test =  
    train_test_split( X, y, test_size=0.30,  
        random_state=42)
```

```
gb_model = GradientBoostingClassifier(  
    n_estimators=100, # number of trees  
    learning_rate=0.1, # shrinkage rate  
    max_depth=3, # depth of each tree  
    random_state=42  
)  
gb_model.fit(X_train,  
    y_train)
```

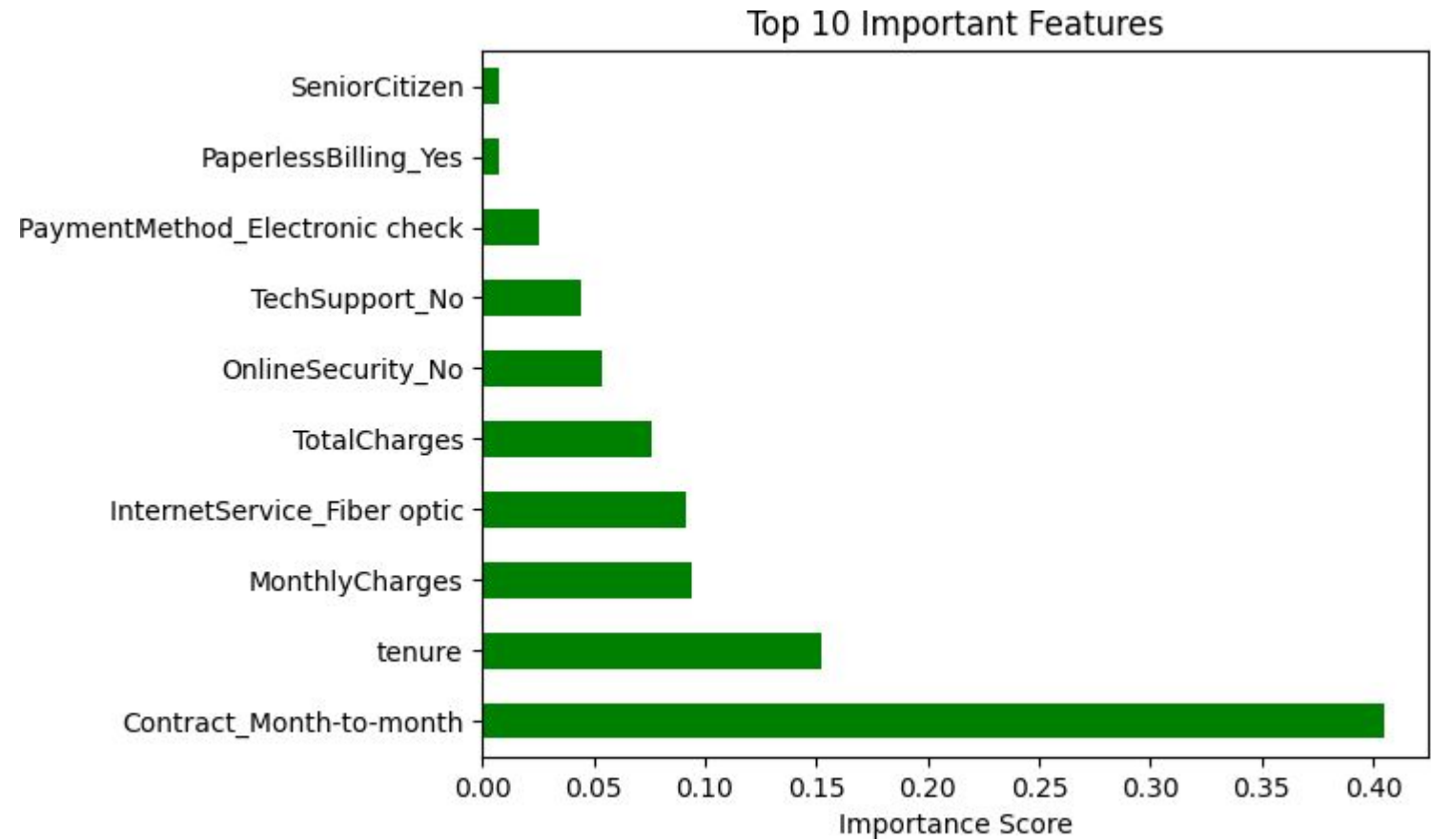
Accuracy: 0.8004739336492891





Gradient Boosting Machines:

```
# Feature importance chart
importances =
pd.Series(gb_model.feature_importances_, index=X.columns)
importances.nlargest(10).plot(kind='barh', color='green')
plt.title("Top 10 Important Features")
plt.xlabel("Importance Score")
plt.show()
```





Gradient Boosting Machines (GBM) Industrial

Finance: Fraud detection, credit scoring, and stock market prediction.

Healthcare: Disease diagnosis, patient outcome prediction.

Retail/Marketing : Customer segmentation, demand forecasting,
recommendation systems.

Emerging Areas: Smart cities (optimizing infrastructure), environmental
modeling, cybersecurity (anomaly detection in network traffic)



Unsupervised Learning Techniques K-Means, Hierarchical Clustering, and DBSCAN, and their use cases in industrial scenarios.

Unsupervised Learning is a type of machine learning where the model learns patterns **from unlabeled data** — meaning there is no target variable provided. Instead of learning a mapping from inputs to outputs (as in supervised learning), it tries to **understand the structure, distribution, and relationships** within the data.

Key Goals of Unsupervised Learning:

- Discover hidden patterns
- Group similar data points together
- Reduce data dimensionality for easier analysis or visualization



Unsupervised Learning

Advantages of Unsupervised Learning:

- No need for labeled data (which is often costly or time-consuming to obtain)
- Can uncover previously unknown patterns
- Useful for exploratory data analysis (K-means)

Limitations Unsupervised Learning :

- Harder to evaluate (no ground truth to compare results to)
- Results can be ambiguous
- Sensitive to parameter choices (e.g., number of clusters in

Unsupervised learning helps uncover hidden patterns in unlabeled data—no target variable. Clustering groups similar data points together. Three core approaches:

K-Means (partitional, centroid-based)

Hierarchical Clustering (tree-based, agglomerative or divisive)

DBSCAN (density-based, identifies arbitrary shapes & noise)



K mean Example

Example: Suppose we want to group the visitors to a website using just their age as follows:

16, 16, 17, 20, 20, 21, 21, 22, 23, 29, 36, 41, 42, 43, 44, 45, 61, 62, 66

Initial Cluster:

$K=2$ Centroid($C1$) = 16 [16] Centroid($C2$) = 22 [22]

Note: These two points are chosen randomly from the dataset.

Iteration-1:

$C1 = 16.33$ [16, 16, 17] $C2 = 37.25$ [20, 20, 21, 21, 22, 23, 29, 36, 41, 42, 43, 44, 45, 61, 62, 66]

Iteration-2:

$C1 = 19.55$ [16, 16, 17, 20, 20, 21, 21, 22, 23] $C2 = 46.90$ [29, 36, 41, 42, 43, 44, 45, 61, 62, 66]

Iteration-3:

$C1 = 20.50$ [16, 16, 17, 20, 20, 21, 21, 22, 23, 29] $C2 = 48.89$ [36, 41, 42, 43, 44, 45, 61, 62, 66]

Iteration-4:

$C1 = 20.50$ [16, 16, 17, 20, 20, 21, 21, 22, 23, 29] $C2 = 48.89$ [36, 41, 42, 43, 44, 45, 61, 62, 66]

No change Between Iteration 3 and 4, so we stop.

Therefore we get the clusters (16-29) and (36-66) as 2 clusters we get using K Mean Algorithm.



Measures the Quality of Clustering(Similarity)

An important issue in clustering is how to determine the similarity between two objects, so that clusters can be formed from objects with high similarity within clusters and low similarity between clusters.

Commonly, to measure similarity or dissimilarity between objects, a distance measure such as Euclidean, Manhattan and Minkowski is used.

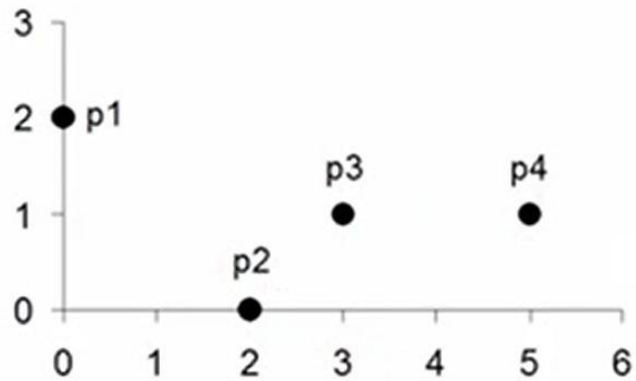
A distance function returns a lower value for pairs of objects that are more similar to one another.

Euclidean $\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$

Manhattan $\sum_{i=1}^k |x_i - y_i|$

Minkowski $\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$

Euclidean Distance



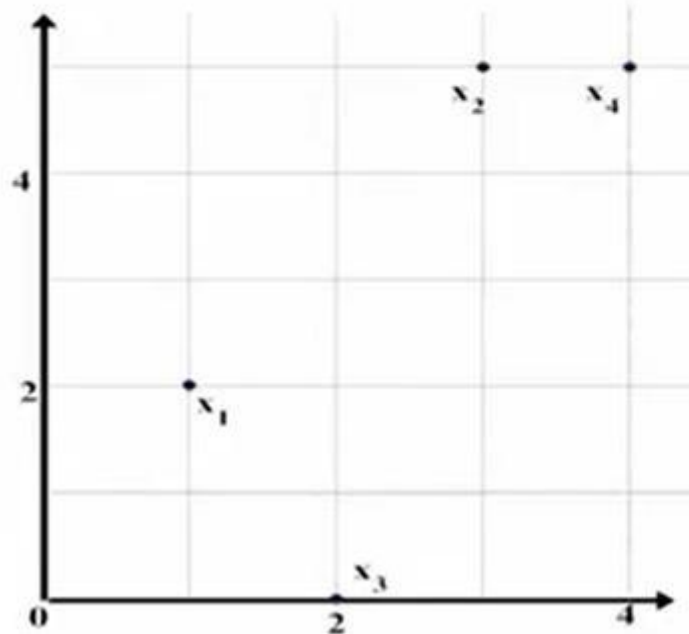
point	x	y
p1	0	2
p2	2	0
p3	3	1
p4	5	1

	p1	p2	p3	p4
p1	0	2.828	3.162	5.099
p2	2.828	0	1.414	3.162
p3	3.162	1.414	0	2
p4	5.099	3.162	2	0

Distance Matrix

Example Minkowsky Distance at Special Cases

point	attribute 1	attribute 2
x1	1	2
x2	3	5
x3	2	0
x4	4	5



Manhattan (L_1)

L	x1	x2	x3	x4
x1	0			
x2	5	0		
x3	3	6	0	
x4	6	1	7	0

Euclidean (L_2)

L2	x1	x2	x3	x4
x1	0			
x2	3.61	0		
x3	2.24	5.1	0	
x4	4.24	1	5.39	0

Supremum (L_∞)

L_∞	x1	x2	x3	x4
x1	0			
x2	3	0		
x3	2	5	0	
x4	3	1	5	0

Step by step Pictorial representation of K-Means Clustering



figure-1



figure-2



figure-3



figure-4



figure-5



figure-6

- ✓ The small circles are the data points, the four ray stars are the centroids (means).
- ✓ The initial configuration is on the figure-1.
- ✓ The algorithm converges after five iterations presented on the figures, from figure-2 to figure-6.



Generate Synthetic Blob Data

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np
```

```
# Create 4 clusters of 500 points
```

```
X, _ = make_blobs(n_samples=500, centers=4,
cluster_std=[1.0, 2.5, 1.5, 0.5], random_state=42)
```

```
# Standardize
```

```
scaler = StandardScaler()
```

```
X_scaled =
```

```
scaler.fit_transform(X)
```

```
# Plot raw blobs
```

```
plt.figure(figsize=(6, 5))
```

```
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], s=10)
```

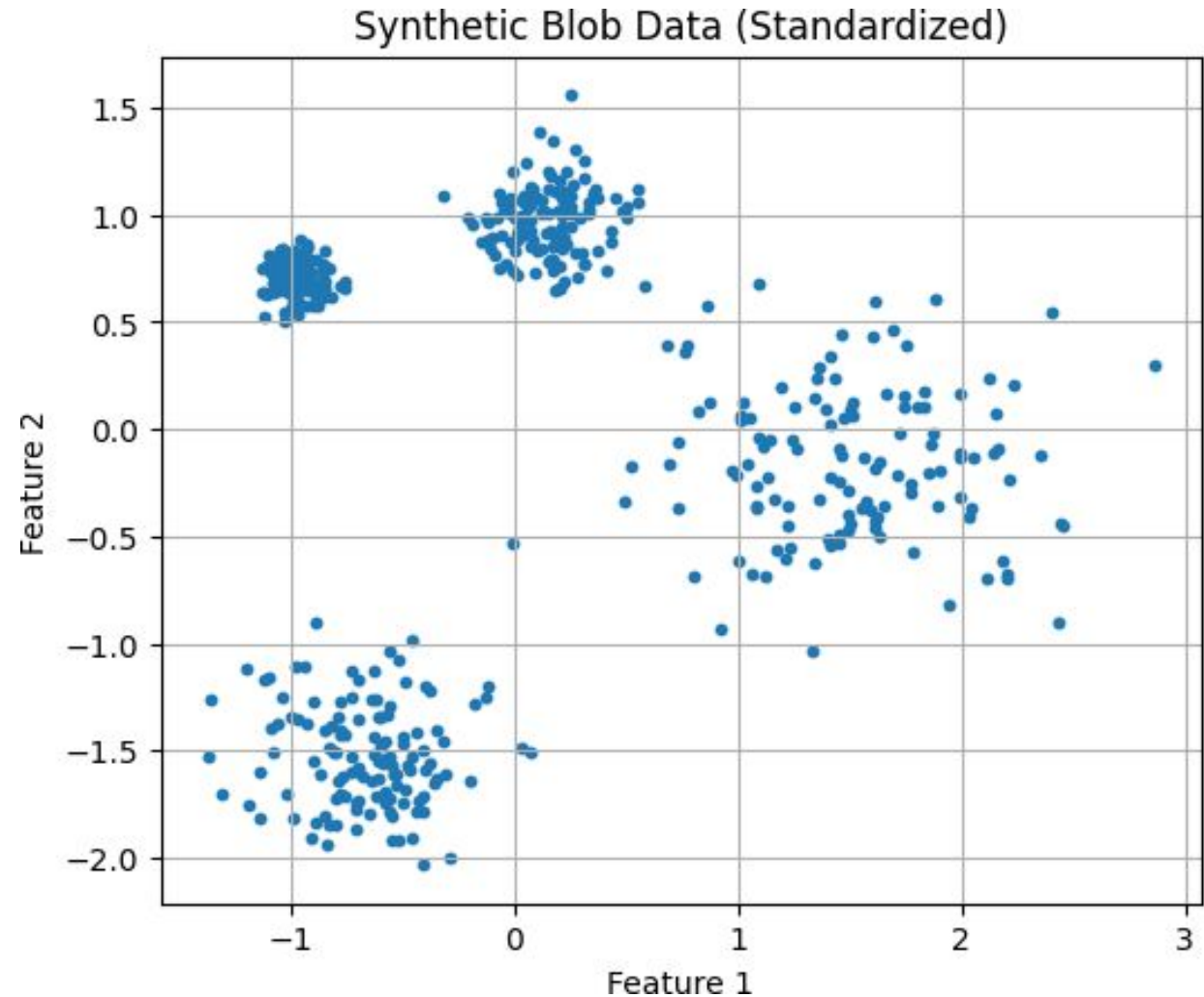
```
plt.title("Synthetic Blob Data (Standardized)")
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.grid(True)
```

```
plt.show()
```





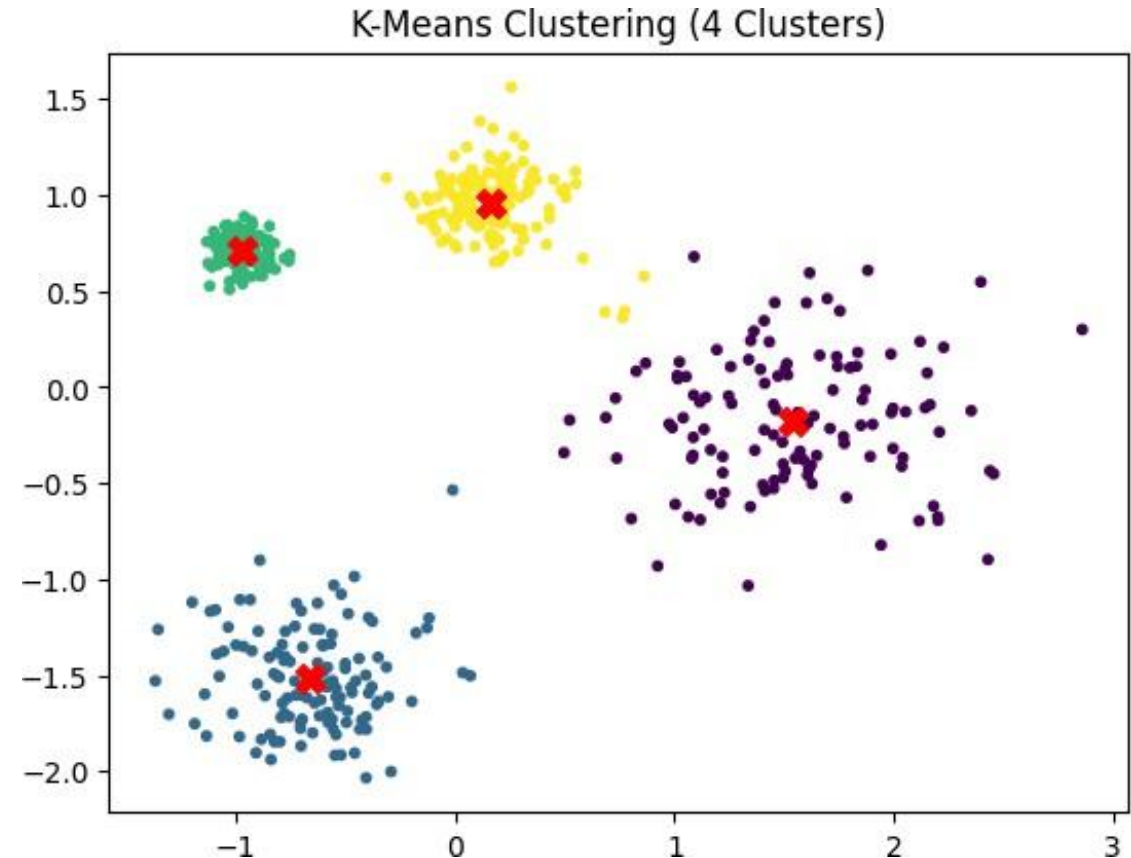
K-Means

K-Means

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
kmeans = KMeans(n_clusters=4,
random_state=42)
labels_kmeans =
kmeans.fit_predict(X_scaled) score_kmeans =
silhouette_score(X_scaled, labels_kmeans)
print("K-Means Silhouette Score:",
score_kmeans)
```

```
plt.scatter(X_scaled[:, 0], X_scaled[:,
1], c=labels_kmeans, cmap='viridis', s=10)
plt.scatter(kmeans.cluster_centers[:, 0],
kmeans.cluster_centers[:, 1], c='red',
marker='X', s=100)
plt.title("K-Means Clustering (4
Clusters)") plt.show()
```



K-Means Silhouette Score:

0.745282213796944



K-Means Industrial Applications:

Customer Segmentation: Differentiating customer groups for targeted marketing.

Retail: Product clustering for inventory and supply chain optimization.

Social Media **bold text:** User grouping based on online behavior.

Bioinformatics: Grouping genes based on expression data.

Document Clustering: Organizing documents for easier retrieval.

Speech Recognition: Grouping acoustic signals for training models



Hierarchical Clustering

Hierarchical Clustering Agglomerative (bottom-up): start with each point as its own cluster, then merge step by step based on distance/linkage criteria to form a dendrogram tree

Hierarchical clustering builds nested clusters by either merging or splitting clusters successively, useful for discovering hierarchical relationships

Two Main Types of Hierarchical Clustering:

Type	Description
Agglomerative (bottom-up)	Starts with each point as its own cluster, then merges them step by step
Divisive (top-down)	Starts with all points in one cluster, then splits them recursively

Agglomerative is more common and widely used.

How Agglomerative Clustering Works:

1. Treat each data point as a separate cluster.
2. Compute the **distance (dissimilarity)** between all clusters.
3. **Merge** the two closest clusters.
4. Repeat steps 2–3 until all points are in one cluster (or until desired number of clusters is reached).
5. Visualize with a **dendrogram** (tree-like diagram).



Hierarchical Clustering

```
# Apply Hierarchical Clustering
```

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
```

```
# Dendrogram (optional visualization)
```

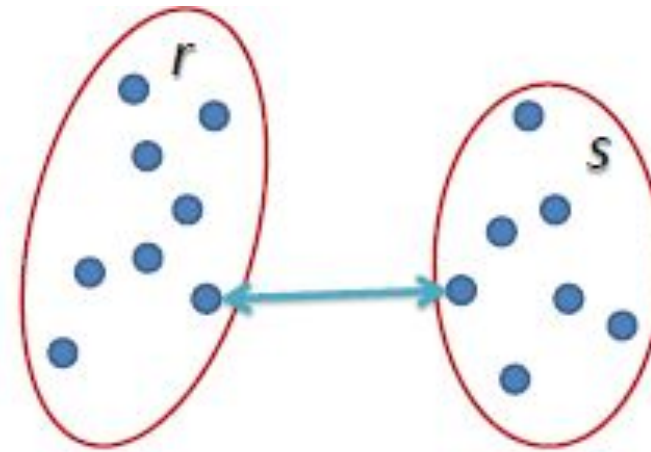
```
linked = linkage(X_scaled,
method='ward') plt.figure(figsize=(10,
4))
dendrogram(linked,      truncate_mode='lastp',
p=20)    plt.title("Hierarchical Clustering
Dendrogram") plt.show()
```

```
# Fit Agglomerative clustering
```

```
hclust = AgglomerativeClustering(n_clusters=4,
linkage='ward') labels_hier = hclust.fit_predict(X_scaled)
```

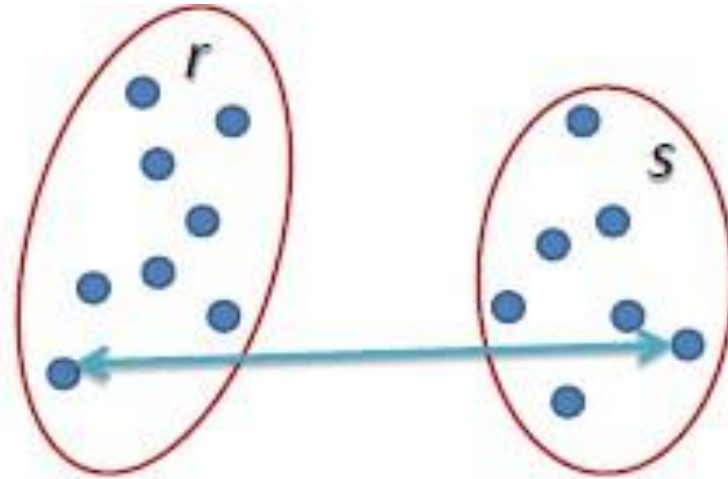
```
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels_hier, cmap='Accent',
s=10)
plt.title("Agglomerative (Hierarchical) Clustering")
plt.show()
```

- **Single Linkage**-In single linkage hierarchical clustering, the distance between two clusters is defined as the *shortest* distance between two points in each cluster. For example, the distance between clusters “r” and “s” to the left is equal to the length of the arrow between their two closest points.



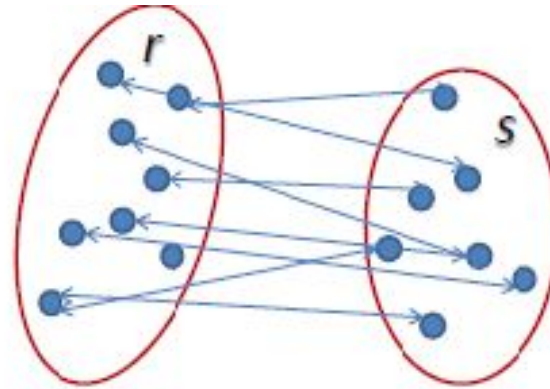
$$L(r, s) = \min(D(x_{ri}, x_{sj}))$$

- **Complete Linkage-** In complete linkage hierarchical clustering, the distance between two clusters is defined as the *longest* distance between two points in each cluster. For example, the distance between clusters “r” and “s” to the left is equal to the length of the arrow between their two furthest points.



$$L(r, s) = \max(D(x_{ri}, x_{sj}))$$

- **Average Linkage**-In average linkage hierarchical clustering, the distance between two clusters is defined as the average distance between each point in one cluster to every point in the other cluster. For example, the distance between clusters “r” and “s” to the left is equal to the average length each arrow between connecting the points of one cluster to the other.



$$L(r, s) = \frac{1}{n_r n_s} \sum_{i=1}^{n_r} \sum_{j=1}^{n_s} D(x_{ri}, x_{sj})$$

- **Ward's linkage:** In this method, all clusters are considered and the algorithm computes the sum of squared distances within the clusters and merges them to minimize it. From a statistical viewpoint, the process of agglomeration leads to a reduction in the variance of each resulting cluster. The measure is:

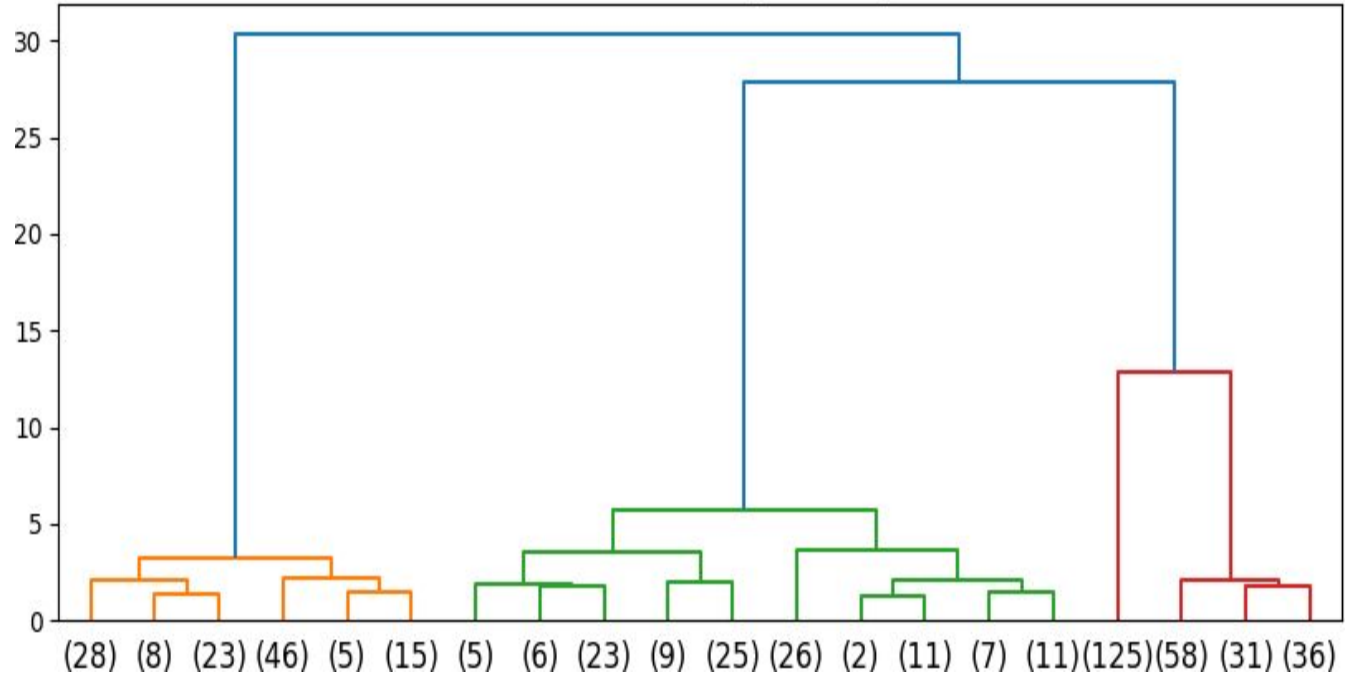
$$\forall C_i, C_j \quad L_{ij} = \sum_{x_a \in C_i} \sum_{x_b \in C_j} \|x_a - x_b\|^2$$

- The Ward's linkage supports only the Euclidean distance.

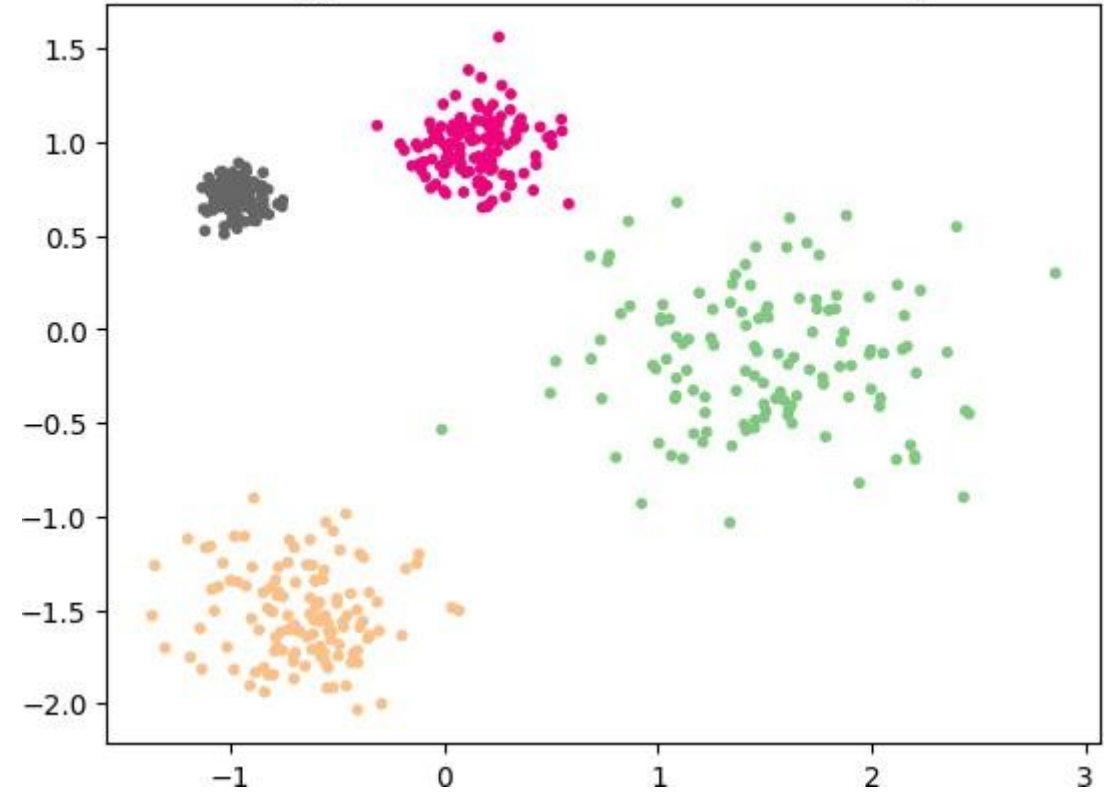
- **Dendrograms**
- To better understand the agglomeration process, it's useful to introduce a graphical method called a **dendrogram**, which shows in a static way how the aggregations are performed, starting from the bottom (where all samples are separated) till the top (where the linkage is complete). Unfortunately, scikit-learn doesn't support them. However, SciPy (which is a mandatory requirement for it) provides some useful built-in functions.
- Let's start by creating a dummy dataset:

Hierarchical Clustering

Hierarchical Clustering Dendrogram



Agglomerative (Hierarchical) Clustering





Hierarchical clustering Industrial Applications:

Customer Segmentation: Discovering audience subgroups for personalized marketing.

Healthcare: Personalizing treatment by identifying phenotype-based patient subgroups.

Finance: Fraud detection by identifying unusual transaction patterns.

Telecommunications: Identifying usage-based customer groups to reduce churn.

Recruitment: Categorizing candidates by skill sets for targeted hiring



DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Clusters defined by dense regions: requires two parameters: ϵ (eps radius) and min_samples for core-point density. Does not assume spherical clusters and automatically finds outliers as noise points.

DBSCAN groups together data points that are closely packed while marking outliers as noise

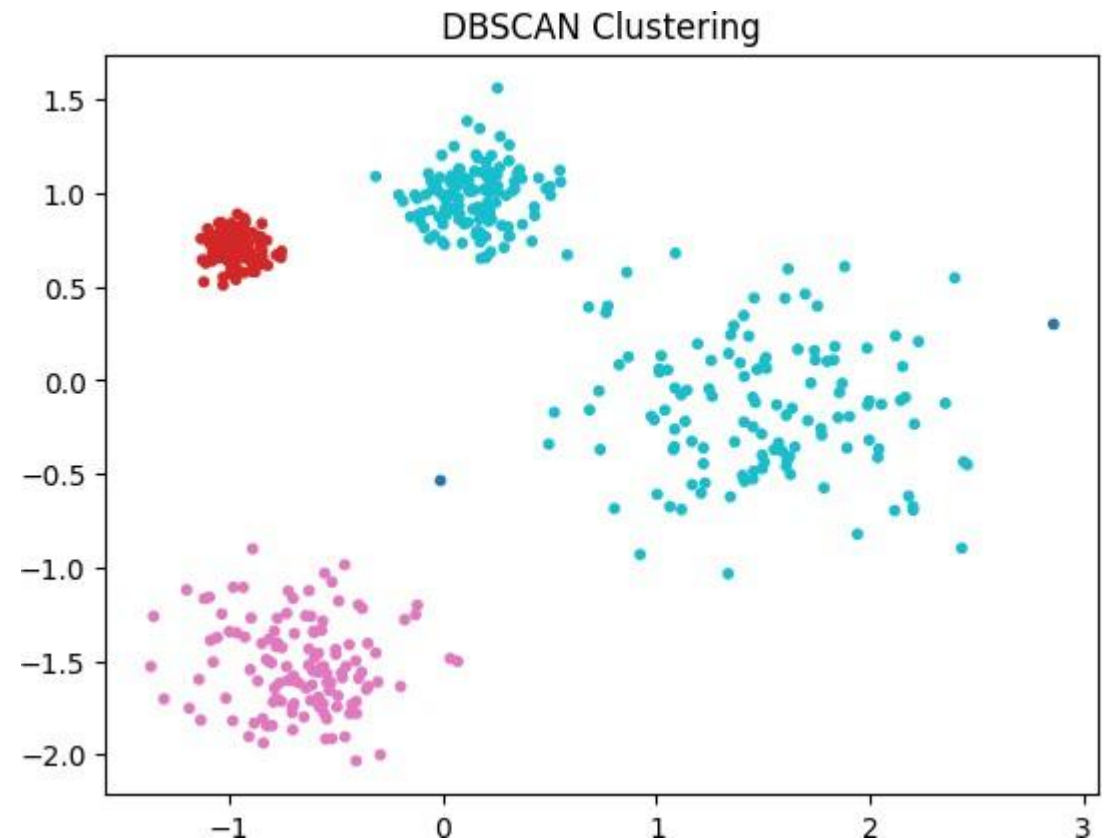
```
# Apply DBSCAN

from sklearn.cluster import DBSCAN

# Try a few eps values to tune
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels_dbscan = dbscan.fit_predict(X_scaled)

plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels_dbscan,
            cmap='tab10', s=10)
plt.title("DBSCAN Clustering")
plt.show()

# Noise detection
n_noise = np.sum(labels_dbscan == -1)
print(f"Number of noise points detected by DBSCAN: {n_noise}")
```





DBSCAN Industrial Applications:

Geospatial Analysis: Identifying urban hotspots for urban planning or public services.

Financial Sector: Fraud detection by isolating anomalous transactions.

Manufacturing/IoT: Sensor data clustering to spot equipment anomalies or failures for predictive maintenance.

Healthcare: Identifying irregular patient data or emergency events in medical sensors.

Image Segmentation: Distinguishing regions (e.g., tumors) in medical imaging.



Comparison Table

Algorithm	Need to Specify K	Cluster Shape	Handles Noise?	Typical Use Cases
K-Means	✓	Spherical / convex	✗	Quick partitioning, simple data
Hierarchical	Optional (dendrogram)	Flexible linkage-based	✗ / manual	Understanding hierarchy structure
DBSCAN	✗ (density params)	Arbitrary shaped	✓	Spatial, irregular clusters, outlier detection



Suggested Readings

Text Books:

1. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron - Covers Unit I & II: Foundational ML concepts, supervised & unsupervised learning, and data preprocessing.
2. Pattern Recognition and Machine Learning" by Christopher M. Bishop - Supports Unit I & IV: Deep dive into mathematical foundations and evaluation techniques.
3. Machine Learning Engineering" by Andriy Burkov - Excellent for Unit III: Practical model deployment, CI/CD, and ML systems in industry
4. Deep Learning with Python" by François Chollet - Addresses Unit V: Includes foundation models, generative AI, and on-device deep learning.

Reference Books:

1. Introduction to Statistical Learning" by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani - Relevant for Units I & II: Statistical methods, linear models, and feature engineering.
2. Designing Machine Learning Systems with Python" by David Julian - Supports Unit III: Implementation, pipelines, and deployment using industry tools
3. Ethics of Artificial Intelligence and Robotics" (Stanford Encyclopedia of Philosophy) - Unit IV: Ethical, societal, and legal implications of ML and AI.
4. Graph Representation Learning" by William L. Hamilton - Unit V: In-depth reference on Graph Neural Networks and their applications.